ASSIGNMENT SUBJECT 501: Linux Operating System

Submission Date: 14/08/2026

1. Explain Shell in Linux.

What is a Shell?

A **Shell** is a **command-line interface (CLI)** that allows users to interact with the Linux operating system.
It reads user input (commands), interprets it, and passes it to the kernel for execution.

Think of it like a **translator** between the user and the system.

Common Types of Shell in Linux

Here are the three popular shells used in Linux:

Shell Name	Full Form / Type
Bash	Bourne Again SHell
Zsh	Z Shell
Dash	DebianAlmquist Shell

1. Bash (Bourne Again Shell)

- Most commonly used shell in Linux systems.
- Default shell in many distributions (e.g., Ubuntu).
- Based on the original Bourne Shell (sh), but with more features.

Features:

- Command history
- Tab completion (auto-completes file names or commands)
- Command aliasing
- Scripting support with variables, loops, conditionals

- Supports keyboard shortcuts (Ctrl+C to stop, etc.)

2. Zsh (Z Shell)

- Advanced shell built on top of Bash features.
- More customizable and user-friendly.

Features:

- Everything in Bash + more
- **Auto-correction** (fixes typos in commands)
- **Theme and plugin support** (e.g., using Oh My Zsh)
- Advanced tab completion
- Better scripting support
- Used by developers and power users

3. Dash (Debian Almquist Shell)

- Lightweight and fast shell.
- Focused on **speed and POSIX compliance** (follows Linux standards strictly).
- Used mostly in **system scripts** and for booting Linux.

Features:

- Faster than Bash and Zsh
- Uses less memory
- No interactive features like auto-complete or history
- Not user-friendly for regular terminal usage

2. Explain Control Flow Structures in details.

Control flow structures help you write **decision-making** and **repetitive** logic in shell scripts. Below are the commonly used structures:

1. if–else Statement

Used for conditional execution of commands.

Syntax:

```
if [ condition ]
then
commands
elif [ another_condition ]
then
commands
else
commands
fi
```

Example:

```
num=5
if [ $num -gt 0 ]
then
echo"Positive number"
else
echo"Zero or negative"
fi
```

2. case Statement

Used as a multi-way decision structure (like switch-case in other languages).

Syntax:

```
case$variablein
  pattern1)
commands ;;
  pattern2)
commands ;;
  *)
default commands ;;
esac
```

Example:

```
echo "Enter a number (1-7):"

read day
```

```
case $day in
  1)
echo "Monday"
    ;;
  2)
echo "Tuesday"
    ;;
  3)
echo "Wednesday"
    ;;
  4)
echo "Thursday"
    ;;
  5)
echo "Friday"
    ;;
  6)
echo "Saturday"
    ;;
  7)
echo "Sunday"
    ;;
```

```
  *)
echo "Invalid input! Please enter a number between 1 and 7."
    ;;
esac
```

3. for Loop

Repeats a block of code for each item in a list.

Syntax:

```
forvarin list
do
commands
done
```

Example:

```
for i in 1 2 3 4 5
do
echo "Number: $i"
done
```

4. while Loop

Runs as long as the condition is **true**.

Syntax:

```
while [ condition ]
do
commands
done
```

Example:

```
i=1
while [ $i -le 5 ]
do
echo"Number: $i"
  i=$((i+1))
done
```

5. until Loop

Runs as long as the condition is **false**.

Syntax:

```
until [ condition ]
do
commands
done
```

Example:

```
i=1

while [ $i -le 5 ]

do

echo "Number: $i"

  i=$((i + 1))

done
```

3. Write a note on Pattern Matching using grep.

grep

The `grep` command is one of the most useful tools in Linux and Unix systems. It is used to search for specific words, phrases, or patterns inside text files, and shows the matching lines on your screen.

Syntax of grep Command in Unix/Linux

The basic syntax of the **`grep`** command is as follows:

```
grep [options] pattern [files]
```

- [**options**]: These are command-line flags that modify the behavior of grep.
- [**pattern**]: This is the regular expression you want to search for.
- [**file**]: This is the name of the file(s) you want to search within. You can specify multiple files for simultaneous searching.

Example of grep Command in Linux

grep Command is useful when you need to quickly find certain keywords or phrases in logs or documents.

Example : Search for a word in a file

If you have a file called `notes.txt` and you want to find all lines containing the word Python, you can use:

```
grep "hello" file1grep .txt
```

Options in grep Command in Linux

Its name comes from an old editor command: *g/re/p*, which stands for "globally search for a regular expression and print. Here are some use cases of grep command:

1. **Case insensitive search**

The -i option enables to search for a string case insensitively in the given file. It matches the words like "UNIX", "Unix", "unix".

```
grep -i "Word" file1.txt
```

2. **Displaying the Count Matches Using grep**

We can find the number of lines that matches the given string/pattern

```
grep -c "word" file1.txt
```

3. **Display the** Matching Filenames **Using grep**

We can just display the files that contains the given string/pattern.

```
grep -l "unix" *
```

or

```
grep -l "unix" file1.txt file2.txt file3.xt file4.txt
```

4. **Checking Whole Words** Using grep

By default, grep matches the given string/pattern even if it is found as a substring in a file. The -w option to grep makes it match only the whole words.

```
grep -w "unix" geekfile.txt
```

5. **Display Matched Pattern** Using grep
By default, grep displays the entire line which has the matched string. We can make the grep to display only the matched string by using the -o option.

```
grep -o "unix" geekfile.txt
```

6. **Show Line Numbers with grep -n**
To show the line number of file with the line matched.

```
grep -n "unix" geekfile.txt
```

7. Inverting the Pattern Match Using grep
You can display the lines that are not matched with the specified search string pattern using the -v option.

```
grep -v "unix" geekfile.txt
```

8. **Match Lines Starting with a String using grep**
The ^ regular expression pattern specifies the start of a line. This can be used in grep to match the lines which start with the given string or pattern.

```
grep "^unix" geekfile.txt
```

9. **Match Lines Ending with a String using grep**
The $ regular expression pattern specifies the end of a line. This can be used in grep to match the lines which end with the given string or pattern.

```
grep "os$" geekfile.txt
```

10. Specifies expression with -e option
Can use multiple times :

```
grep –e "Agarwal" –e "Aggarwal" –e "Agrawal" geekfile.txt
```

11. **Use -f to Read Patterns from a File**
It is used to search for multiple patterns, listed in a separate file, within another target file.

```
cat pattern.txt
```

Agarwal
Aggarwal
Agrawal

```
grep –f pattern.txt  geekfile.txt
```

12. **Print Specific Lines from a File with grep**

-A prints the searched line and n lines after the result, -B prints the searched line and n lines before the result, and -C prints the searched line and n lines after and before the result.

Syntax:

```
grep -A[NumberOfLines(n)] [search] [file]
grep -B[NumberOfLines(n)] [search] [file]
grep -C[NumberOfLines(n)] [search] [file]
```

Example:

```
grep -A1 learn geekfile.txt
```